

ЛАБОРАТОРНАЯ РАБОТА № 6

ВВОД И ВЫВОД РАЗЛИЧНЫХ ТИПОВ ДАННЫХ. ПОСТРОЕНИЕ ПРОСТЫХ ГРАФИЧЕСКИХ ПРИМИТИВОВ С ПОМОЩЬЮ ЛИСП-ПРОГРАММЫ

Цель работы: изучение различных типов данных *Visual LISP*, способов ввода и вывода простых графических примитивов.

1. ТЕОРЕТИЧЕСКИЙ МАТЕРИАЛ

1.1. Типы данных в *Visual LISP*

LISP работает с данными следующих типов:

- ❖ целое число;
- ❖ вещественное число;
- ❖ строка;
- ❖ список, точечная пара, точечный список;
- ❖ дескриптор файла;
- ❖ указатель функции, приложения;
- ❖ примитив *NanoCAD*;
- ❖ набор.

Символам *LISP* могут быть присвоены значения любых типов. Символ языка *LISP* может в ходе программы менять тип своего значения. Пользователь в явном виде может задавать константы следующих типов: целые, вещественные, строковые и списковые.

Рассмотрим подробно правила работы с различными данными (атомами) в *Visual LISP*: целыми и действительными константами, текстовыми (строковыми) константами, а также переменными (символическими именами, идентификаторами).

Целая константа задается правилом:

$\langle \text{целая константа} \rangle ::= [+/-] \langle \text{цифра} \rangle \dots$

Здесь $+/-$ означает плюс или минус. Целые числа могут задаваться в пределах от -32768 до 32767. Целое число без знака – от 0 до 65535.

Действительная константа задается правилом:

$\langle \text{действительная константа} \rangle ::= [+/-] \langle \text{цифра} \rangle \dots . \langle \text{цифра} \rangle \dots$

Здесь точка отделяет целую часть от дробной части.

Для чисел с большими порядками удобна запись

$[+/-] \langle \text{цифра} \rangle \dots \langle \text{цифра} \rangle E [-] \langle \text{цифра} \rangle [\langle \text{цифра} \rangle]$.

Здесь *E* означает «10 в степени», а показатель степени ставится после *E*.

В *Visual LISP* определена также константа, обозначаемая *PI* и имеющая значение 3.1415926.

Константу *PI* можно назвать предопределенным символом. К предопределенным символам в *LISP* относятся следующие:

- ❖ *T* – единственная логическая константа, имеющая значение истина;

- ❖ *NIL* – пустое значение, не эквивалентное числу 0 или пустой строке “”;
- ❖ *PI* – постоянная, равная отношению длины окружности к диаметру;
- ❖ *PAUSE* – постоянная, позволяющая ввести паузу при работе команды графического редактора (используется в функции *COMMAND*).

Значения предопределенных символов пользователям не рекомендуется изменять.

Для текстовой константы можно использовать запись:

< текстовая константа > ::= “< символ >...”.

Длина текстовой константы не более 100 знаков. Если в тексте встречается знак «\», то он должен быть удвоен (т.е. вместо «\» должно быть записано «\\»). Это связано с тем, что символ «\» используется для ввода управляющих символов: \n – перевод строки, \r – возврат каретки, \t – табуляция.

Для переменной используется запись:

< переменная > ::= < символ >...

Переменные формируются из любого количества символов, кроме знаков (,), точка, двоеточие, пробел, перевод строки. Первым символом должна быть буква. Переменная – это объект, имеющий имя и значение. Имя служит для обозначения объекта. Правила записи переменных, приведенные выше, это, по сути дела, правила записи имен. Конкретное значение переменная приобретает в процессе выполнения программы, например, переменная, имеющая имя *XX*, может последовательно принимать значения 10, 12.3, «НАЧАЛО», *NIL* ...

Помимо перечисленных атомов определен еще «пустой атом», обозначаемый как *NIL*, что означает «ничего», т.е. атом, не имеющий никакого значения.

Примеры правильной записи данных:

0, -123, 0.0 – целые константы;

12.3, -12.34E8 – действительные константы;

“ABCD”, “Укажите точку”, “\nУкажите точку” – текстовые константы;

AI, *ABCD*, *CROSS_POINT_2* – переменные (имена переменных);

NIL – пусто.

Примеры неправильной записи данных:

-100000 – целая константа за пределами допустимых значений;

.25 – действительная константа без целой части;

25. – действительная константа без дробной части;

“AB\CD” – управляющий символ “\C” не определен, символ “\” должен повторяться дважды;

“ABCD” – отсутствуют закрывающие кавычки;

AB CD – такая запись возможна, но это уже не имя переменной, т.к. внутри имеется пробел.

1.2. Построение простых графических примитивов

Любая команда графического редактора может быть включена в программу на *LISP*, что позволяет строить изображения из программы автоматиче-

ски или с участием пользователя. Это делается с помощью функции *COMMAND*, имеющей вид

(COMMAND "< имя команды >" [< аргумент >...])

Здесь *< имя команды >* – имя требуемой команды, определенной в данной версии графического редактора. Функция *COMMAND* является очень важной функцией, которая имитирует ввод пользователя в командной строке с клавиатуры.

Если командой предусмотрены аргументы, то их значения перечисляются. Аргументами могут быть координаты точек, опции и др. Аргументы перечисляются в той последовательности, в которой Вы задавали бы их, формируя команду для графического редактора в режиме диалога. В качестве аргументов могут использоваться выражения *LISP*. При этом необходимо, чтобы значение, возвращаемое выражением, соответствовало тому значению, которого требует данная команда графического редактора. В качестве аргументов могут быть заданы переменные, если значения этих переменных были ранее определены в программе, или непосредственно заданы значения. Такие значения необходимо заключать в кавычки.

Слово *PAUSE* в качестве аргумента обозначает переключение в режим графического диалога, т.е. после чтения *PAUSE* команда ожидает одного аргумента (опцию, точку и др.) с клавиатуры, от «мыши» и т.д., в точности так, как если бы команда выполнялась просто в графическом редакторе.

Перечислим дополнительные сведения о функции *COMMAND*:

в одной функции может вызываться не одна команда, а последовательность команд;

(COMMAND "") соответствует вводу пробела с клавиатуры;

(COMMAND) равносильно нажатию клавиши *CTRL/C*.

Примеры:

(COMMAND "LINE" "1,1" "10,10" "")

; Эта функция строит линию по точкам 1,1 и 10,10

(COMMAND "CIRCLE" "2P" "10,10" "20,20")

; Строится окружность по точкам на концах диаметра

(COMMAND "CIRCLE" "10,10" PAUSE "LINE" "10,10" "10,5")

; Строится прямоугольник

(COMMAND "LINE" "0,0" "210,0" "210,297" "0,297" "CLOSE") .

В последнем примере строится окружность с центром в точке 10,10 и с радиусом, задаваемым пользователем, затем строится ломаная линия: две первые точки ее задаются из программы, а для задания последующих точек предполагаются действия пользователя, как при выполнении команды *LINE*.

Любую из этих функций можно набрать в ответ на приглашение графического редактора, и соответствующая команда будет выполнена.

Обратим внимание на то, как вводятся координаты точки в функции *COMMAND*: координаты *X* и *Y* отделяются запятой, и все эти записи заключаются в кавычки:

(COMMAND "CIRCLE" PC R).

В этой функции точка центра и радиус окружности заданы переменными. Такая строка не может быть выполнена отдельно, а только в составе программы, где перед данной функцией выполняются другие функции, определяющие конкретные значения *PC* и *R*. Такие программы будут рассмотрены ниже.

Если при выполнении функции какой-либо аргумент представлен переменной, то этой переменной должно быть ранее присвоено значение (число, строка текста, список и др.). Основное средство для присвоения значений переменных – функция *SETQ*:

(SETQ < переменная1 > < выражение1 > [< переменная2 > < выражение2 >]...).

Функция действует следующим образом: выполняются действия, задаваемые выражением 1, и результат, возвращаемый этим выражением, становится текущим значением переменной 1. Если имеются другие «пары», то с ними функция поступает аналогично. Функция *SETQ* возвращает значение последнего выражения.

Функция *SETQ* может использоваться с любым количеством аргументов, которое должно быть обязательно четным и не менее двух. Функция *SETQ* – основное средство для сохранения значений, возвращаемых другими выражениями.

Для хранения данных пользователь может вводить свои символы (переменные), не совпадающие по написанию с зарезервированными (предопределенными) или ранее занятыми. Создание новых переменных осуществляется с помощью функции *SETQ*, например:

(SETQ r1 15.33 s24 9) .

Здесь вводятся переменные *r1* и *s24*, получающие соответственно, значения 15.33 (вещественное) и 9 (целое). При этом к той части оперативной памяти, которая отведена для текущего рисунка, добавляются участки, занимаемые переменными. Если переменной присвоить значение *NIL*, то такая переменная из памяти удаляется, и ее место освобождается для других операций *LISP*. Переменные могут использоваться в любых выражениях.

Примеры:

(SETQ A 10.0) ; записывает 10 в A

(SETQ B (+ 123)) ; записывает в B результат сложения 1, 2 и 3, т.е. 6

(SETQ A 1 B 2 C 3) ; записывает 1 в A, 2 в B, 3 в C, возвращает значение 3

(SETQ D (SETQ A 1 B 2 C 3)) ; записывает 3 в D.

Любую из этих функций можно набрать на клавиатуре в ответ на приглашение графического редактора.

Для ввода значений переменных пользователем служат функции типа *GET*. К ним относятся: *GETINT* – ввод целого числа; *GETREAL* – ввод вещественного числа; *GETSTRING* – ввод строки текста; *GETPOINT* – ввод точки; *GETDIST* – ввод расстояния; *GETANGLE* – ввод угла и некоторые другие функции.

Работу функций *GETINT*, *GETREAL*, *GETSTRING* рассмотрим на примере функции *GETINT*, которая в общем виде выглядит так:

(GETINT [< подсказка >]).

Эта функция создает паузу в выполнении программы и ожидает, пока пользователь введет с клавиатуры целое число. Функция возвращает введенное число. Необязательный аргумент *< подсказка >* – это текстовая константа. Если этот аргумент присутствует, то во время паузы на экране будет высвечиваться текст подсказки.

Рассмотрим пример работы с функцией *GETINT*.

Пример:

(SETQ NUM (GETINT “Введите номер детали <I>: ”)) .

Здесь функция *GETINT* выводит на экран запрос “Введите номер детали *<I>*: ”. Ее возвращаемым значением будет *NIL*, если пользователь ответит простым нажатием клавиши *<Enter>*, или введенное пользователем допустимое целое число (например, 21). В случае ввода пользователем недопустимого целого числа (например, -5) функция *GETINT* выдаст сообщение об ошибочном значении и будет ожидать допустимого варианта ввода. [2]

Аналогично записываются и действуют функции *GETREAL* и *GETSTRING*.

Запишем правило для функции *GETPOINT*:

(GETPOINT [< точка >] [< подсказка >]).

Функция создает паузу и ожидает ввода точки. Точка может быть введена путем указания на экране или путем набора ее координат на клавиатуре в виде *X, Y*. Функция возвращает координаты точки (в виде списка). Аргумент *< подсказка >* имеет тот же смысл, что и для функций *GETINT*. Если задан аргумент *< точка >*, то при перемещении курсора на экране будет отображаться «резиновая линия» от данной «опорной» точки до текущего положения курсора.

Функция *(GETDIST [< точка >][< подсказка >])* создает паузу и ожидает ввода расстояния. Расстояние может быть указано тремя способами: как действительное число с клавиатуры; путем указания двух точек (будет определено расстояние между точками); путем указания второй точки при наличии аргумента *< точка >*. В двух последних случаях между первой точкой и текущим положением курсора рисуется «резиновая линия». Функция возвращает действительное число, равное введенному расстоянию. Нетрудно видеть, что первый способ задания расстояния для пользователя (и для программы) будет эквивалентен действию функции *GETREAL*.

Аналогично записывается и действует функция *GETANGLE*, которая возвращает угол (в радианах) между прямой, заданной двумя точками, и осью *X*.

Рассмотрим программу, текст которой приведен ниже.

Программа 2. Построение окружности.

(DEFUN OKR; Имя описываемой функции

(SETQ PC (GETPOINT “\nВведите центр: ”))

(SETQ R (GETDIST PC “\nВведите радиус: ”))

(COMMAND “CIRCLE” PC R))

Такую программу можно записать на диск, затем в любое время вызвать с диска и запустить на выполнение, набрав (*OKR*) в ответ на приглашение графической системы. Для вывода информации на экран можно использовать функции (*PRIN (C, T)*) в различных модификациях.

2 ПЛАН ОТЧЕТА

1. Титульный лист.
2. Цель работы.
3. Задание.
4. Описание программы.

3 ИНДИВИДУАЛЬНЫЕ ЗАДАНИЯ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ № 6

1. Постройте линию, введя данные точек $T1$, $T2$, $T3$ с использованием функций типа $SETQ$. Точка $T1$ при $X=20+n$, $Y=50+n$; точка $T2$ при $X=80-n$, $Y=100-n$. Используя полярные координаты, постройте точку $T3$ ($R=100-n$, $L=40+n$).
2. Постройте окружность на основе введенных данных точки центра PC с $X=100-n$, $Y=100+n$ и радиуса $R=70+n$.
3. Запишите в переменную K результат $A*B*C$ при $A=15+n$, $B=5+n$, $C=5+n$ с помощью функции $SETQ$.
4. Оформите ввод A , B , C через функцию $GETINT$ или $GETREAL$.
5. Выведите данные на экран для всех заданий.

Для каждого индивидуального задания $n=50+\text{номер студента по списку группы}$.